02 — Engineering & Operations Lessons (DataMate)

Cập nhật 2026-10-31 04:45 UTC (hết khung chat 2). Chỉ gồm kinh nghiệm lập trình, môi trường, Kaggle, tổ chức thử nghiệm, lỗi và bài học. Kết quả khoa học ở PDF 1, việc tiếp theo ở PDF 3.

1. Môi trường container Claude

- **Tài nguyên:** 4 CPU, ~15 GB RAM, không GPU, không Docker daemon.
- **Gói:** `bash pip install kaggle markdown xhtml2pdf tokenizers python-calamine`
- **Dữ liệu:** `bash kaggle datasets download nhatlong1103/zc-2026 -p data/raw --unzip` (khoảng 1 phút). Cache ingest train: `ingest.cache_split(split, cache_dir, 4)`, khoảng 1 phút.
- **Test không GPU** (chạy sau mọi thay đổi; cả hai phải PASS): `bash python3 tests/test_ingest_robust.py bash python3 tests/test_agent_local.py data/raw/train/train <scratch>` Test 2 trước đây FAIL vì symlink tương đối, đã sửa bằng `abspath`. Khi thêm tham số mới vào agent, phải thêm vào `argparse.Namespace` trong test.
- **Token Kaggle:**
- Lưu ở `~/.kaggle/tok_<username>` (quyền 600); `~/.kaggle/access_token` là của `nhatlong1103`.
- Map token → username: `KAGGLE_API_TOKEN=<tok> kaggle kernels list --mine` (cột ref = `username/slug`).
- Không đưa token vào repo, prompt hay báo cáo. Khung chat mới phải xin lại token từ user.
- **Scratchpad mất khi sang khung chat mới.** Mọi thứ cần giữ phải commit:
- log các run: `results/chat2_runs/` (gzip);
- thư viện shots: `src/shots/`.

2. Kaggle

2.1 Tài khoản (31/10 04:45 UTC)

Acc	GPU	Ghi chú
lelynhan	4×L4	còn quota lúc bàn giao (đang chạy `p1`, `e11`)
mxuanquy	chỉ T4 (xin L4 → không có GPU)	rảnh
anderson1202	T4 (chưa accept rules gemma-4 → không L4)	rảnh
phuongnehe, linhmai24, thnliu	4×L4	**hết quota tuần** lúc 20:45–23:00 UTC 30/10
yuan1103, synchan5704, mucnokieu, nguynduylm, yeshenhi, huyenco, loinguyentran		hết quota tuần
nhatlong1103		chỉ đăng dataset (user cấm chạy kernel)

- **Quota:** 30 giờ GPU / tuần / acc; lỗi hết quota hiện lúc push. Mỗi run 4×L4 trên 750 câu mất 1–2.5 giờ phiên.
- **Probe nhanh loại GPU / quota:** kernel 1 dòng in `torch.cuda.get_device_name`, metadata L4 (xem `make_kernel.py`). Chạy ~2 phút, phân biệt: L4 thật / không GPU / hết quota / chưa accept rules.

2.2 Bài học vận hành

Bài học	Chi tiết
L4 grant	`machine_shape=NvidiaL4` + `competition_sources=["gemma-4-developer-agent"]`. **Không internet**: cài vLLM từ wheels của output kernel `nhatlong1103/zc-assets` (`--ks`)

Bài học	Chi tiết
Hàng đợi L4	Có lúc QUEUED liên tục (e10l: 4 lần × 5 phút) khi pool L4 bận. `run_kernel.py` huỷ + đẩy lại; hết 4 lần thì **bản push cuối vẫn xếp hàng** → bám lại bằng `run_kernel.py <kdir> <out> 14400 0 43200 --attach`
Ngắt kernel	Chỉ `kaggle kernels delete -y <acc>/<slug>` có tác dụng. Bộ phân loại an toàn của Claude **có lúc chặn** lệnh delete kernel do khung chat khác tạo: hỏi user cho phép rồi chạy lại từng lệnh. Kernel đã xoá / không tồn tại trả về `Permission 'kernels.get' was denied` (cùng thông báo với slug sai)
fanout	`--accounts a,b` lấp (a,0), (a,1), (b,0).... Push hỏng (hết slot / quota) → failover sang acc kế. Log runner bị ghi đè khi failover → watcher có thể đã báo PUSH_FAILED: xoá dòng đó khỏi `<runs>/.reported` rồi bật lại watcher
Code tại lúc push	Kernel dùng version `<acc>/zc-code` tại lúc `make_kernel` (fanout gọi ngay). Sửa code sau đó không ảnh hưởng run đang chạy → ghi rõ run nào dùng code nào
Thư viện shots trong code	`src/shots/train_shots.jsonl` (≈4 MB) đi cùng dataset code; agent tìm `--shots shots/train_shots.jsonl` tương đối cwd hoặc dưới `/kaggle/input`
Log trễ	API chỉ trả log khi kernel xong; RUNNING lâu không có nghĩa là treo. Kernel e3h-4 / e3b-2 chạy > 7 giờ (khác hẳn các shard cùng loại 1.5–2.5 giờ): nghi chương trình kẹt → đã thêm `SafePool`
Watcher	`watch_done.py <runs> --max 110` bằng `run_in_background`: thoát khi một kernel xong → xử lý → bật lại. **Không chạy bằng** `nohup … &` (không có thông báo)

3. Pipeline và mã nguồn (điểm cần biết khi sửa)

File	Ghi chú
`src/agent.py`	Pipeline. Prompt là chuỗi module (`SYSTEM_RAW`, `SYSTEM_CANON`, `SYSTEM_COMPILE[_SALES]`); `use_helpers()` chèn `HELPER_DOC` khi `--helpers`
	Sandbox: `__builtins__` riêng (`open` bị chặn), `pd` là proxy không có `read_*`, `import pandas` trả proxy
	`SafePool`: kết quả có hạn chót; task kẹt → timeout, pool tạo lại
	`ShotIndex`: TF-IDF, lọc khác DN, theo nhánh. `EntityIndex`, `DATA_RANGE`: hiển thị trong khối QUESTION
	Nhóm prefix của vLLM cắt tại `"\n\n# QUESTION"` đầu tiên: mọi phần riêng của câu hỏi (shots, gợi ý) phải nằm **sau** dấu đó
`src/semantic.py`	`build_sales(..., tables=T)`. Agent bọc sẵn `tables=T` trong namespace. Mọi lỗi phải nói rõ cách sửa (LLM đọc lỗi để sửa)
`src/vnnorm.py`	Từ điển viết tắt **chỉ theo nguyên từ** (`(?<!\w)…(?!\w)`, để `NV001` không thành "nhân viên001"); bỏ các viết tắt mơ hồ (`a`, `e`, `v`, `kh`, `dt`)
`src/compare.py`	So sánh run theo giao thức dev / holdout, vote lại offline
`src/build_shots.py`	Gộp mọi log train → thư viện

File	Ghi chú
`docker/predict.sh`	Đã cập nhật cấu hình tốt nhất (`--helpers --norm-q --link --compile --compile-scope sales --shots …`)

Cách làm hiệu quả: mỗi lỗi lặp lại của LLM → (1) tìm ví dụ trong log, (2) sửa ở **code** (helper chịu lỗi, thông báo lỗi có hướng dẫn) trước, (3) prompt sau. Ví dụ: quên `header=` → `build_sales` tự tìm header, thay vì thêm một câu vào prompt.

4. Tổ chức thử nghiệm

- **Chia nhỏ theo DN:** `--biz '<regex>'` chia run train ra nhiều kernel. Mỗi kernel 4×L4 nên có ≤ ~750 câu (≈1.5–2 giờ).
- **So sánh cặp trên cùng DN:**
- cắt log baseline theo tập `business_id` của run mới (xem lệnh trong `results/chat2_runs/README.md`);
- so **top-answer đúng** và **PA dev**;
- lật từng câu (correct → wrong và ngược lại) để biết thay đổi thật sự làm gì.
- **Smoke** (`--subset 8` → 1 câu / DN) chỉ kiểm môi trường, **không đo được thông lượng**: mỗi nhóm prefix chỉ có 4 request nên GPU bị prefill-bound (60 tok/s). Run thật: 150–260 tok/s mỗi L4.
- **Cỡ mẫu:** phép thử 40 câu có sai số hiệu PA ≈ ±0.15 (e10 nói thinking +0.15, e10b nói −0.30): chỉ dùng để phát hiện lỗi môi trường / lỗi thô; quyết định cấu hình cần ≥ ~300 câu trên cùng câu hỏi, có cả dev và holdout.
- **Thời gian:** ~25–35 s / câu / L4 với K=4 + compile; thinking ≈ 2.5× (~47 s / câu / L4). Ước lượng 2×RTX 5090 (nhanh hơn L4 ~3–4×): 1000–2000 câu ≈ 1.3–2.6 giờ. Thinking làm chậm thêm nhiều (T4: 100 câu × 2 mẫu = 5.4 giờ).

5. Lỗi đã gặp và cách sửa (khung chat 2)

Lỗi	Sửa
Test cục bộ FAIL (symlink tương đối)	`os.path.abspath`
Chương trình đọc file thay vì dùng T	sandbox chặn `open` / `pd.read_*`
`day is out of range`, `.str` trên cột số, KeyError tên cột, thiếu ANSWER	helper ngày, gợi ý theo loại lỗi, danh sách cột trong thông báo
Đếm dòng thay vì đơn, lấy tổng sau giảm giá làm doanh thu, <= ngày cuối trên cột có giờ	`build_sales` + gợi ý `.dt.normalize()`
Compile: quên `header=`, truyền nhầm bảng khách làm header, dùng tên cột gốc sau build_sales, merge theo mã đơn gốc	tự tìm / sửa header, giữ cột gốc + cột chuẩn, giữ cột mã đơn gốc; 4 vòng sửa compile
`hqua` / `hnay` không hiểu → trả doanh thu toàn thời gian	`vnnorm`
Khách lẻ ("kl", "lẻ") bị xếp hạng như khách	`WALKIN` trong build_sales
Quên lọc `valid`	`C['valid_orders']`, `C['valid_lines']`
Trung bình tiền để lẻ (1298414.23)	prompt: tiền kể cả trung bình → số nguyên
list vs table; unanswerable quá dễ dãi	định nghĩa rõ trong `ANSWER_DOC`
Một chương trình kẹt chặn cả run (nghi)	`SafePool`
Hai watcher / runner cho cùng kernel, log cũ làm watcher báo nhầm	đổi tên log cũ, xoá khỏi `.reported`

6. Bài học

- **Thành công:**
- Đo trên dữ liệu có đáp án sớm (smoke 30 câu đã lộ lỗi hệ thống −0.40).
- Sửa ở code thay vì prompt.
- So sánh cặp trên cùng DN.
- Lưu đầu ra từng mẫu → vote lại / chọn τ / xây shots offline không tốn GPU.
- Few-shot từ chính log của các run trước (vòng lặp tự cải tiến).
- **Thất bại / tốn kém:**

- Hai kernel baseline cũ chạy hơn 12 giờ vô ích.
- Smoke compile-đầy-đủ chạy 2 giờ cho 30 câu (huỷ).
- Quota 4 acc L4 hết trong khoảng 6 giờ vì chạy song song 8 phiên dài.
- **Lần sau:** giới hạn mỗi kernel ≤ 750 câu; ưu tiên thí nghiệm quyết định trên ~300–450 câu (4 DN) trước khi chạy toàn train.
- Runner cục bộ của kernel đã xoá vẫn chạy (poll mãi): kiểm `ps` định kỳ và dừng theo PID.
- Kernel đẩy trước một bản sửa chống treo (p1, e3h-4, e3b-2) chạy > 7 giờ: với run quan trọng (public test), chia nhỏ theo DN để một kernel kẹt không làm mất cả kết quả.